

STACKLY KIDS LEARNING PLATFORM

Technical Documentation

Prepared By: Dhamodharan P

Project Type: Responsive Kids Learning, Preschool & Academy Platform

Architecture: Static Multi-Page Frontend with Role-Based Student, Teacher and Admin Dashboard

Version: 1.0

Documentation Date: August 2026

1. Project Overview

1.1 Project Description

Stackly Kids Learning Platform is a responsive educational website designed for a preschool and academy environment. The platform provides children and parents with access to information about courses, teachers, school events, educational resources, blogs, and academy activities.

The project combines a public-facing educational website with an interactive role-based portal. The dashboard system supports three user roles:

- Student / Parent
- Teacher / Faculty
- Administrator

The application is implemented as a frontend-focused web platform using HTML5, CSS3, JavaScript, Tailwind CSS, and reusable JavaScript components. The project also includes interactive learning functionality such as drawing tools, sound effects, ABC learning, number learning, and a balloon-pop mini game.

1.2 Project Objectives

The main objectives of the project are:

- Develop a child-friendly educational website.
- Provide information about preschool courses and programs.
- Display faculty and educator profiles.
- Showcase school events and activities.

- Provide educational blogs and resources.
 - Provide student and parent portal functionality.
 - Provide teacher classroom management functionality.
 - Provide administrator management functionality.
 - Implement interactive learning activities.
 - Provide a shopping/cart experience for STEM learning toys.
 - Create a responsive interface for desktop, tablet, and mobile devices.
 - Maintain reusable JavaScript components and centralized application data.
-

2. Technology Stack

2.1 HTML5

HTML5 is used to create the structure and semantic content of the application.

Major uses include:

- Page structure
 - Navigation
 - Forms
 - Course sections
 - Teacher profiles
 - Event cards
 - Dashboard layouts
 - Footer sections
 - Authentication pages
-

2.2 CSS3

CSS3 is used for custom visual styling and application-wide behavior.

The project includes:

- Custom CSS variables
- Responsive layouts
- Animations
- Custom scrollbar
- Background patterns
- Typography configuration
- Custom components

- Responsive media queries

The primary stylesheet is:

```
assets/css/style.css
```

2.3 Tailwind CSS

The project uses the Tailwind CSS CDN:

```
<script src="https://cdn.tailwindcss.com"></script>
```

Tailwind utility classes are extensively used for:

- Spacing
- Colors
- Typography
- Flexbox
- Grid layouts
- Borders
- Shadows
- Responsive design
- Buttons
- Cards
- Dashboard layouts

This allows the application to build responsive interfaces without creating separate CSS classes for every UI element.

2.4 JavaScript

JavaScript provides the application's interactive functionality.

The project uses multiple JavaScript modules:

```
assets/js/data.js  
assets/js/main.js  
assets/js/modals.js  
assets/js/dashboard.js  
assets/js/interactive-pad.js
```

JavaScript is responsible for:

- Dynamic content
 - Authentication
 - Dashboard rendering
 - Course data
 - Teacher data
 - Event data
 - Shopping cart
 - Search
 - Modals
 - Notifications
 - Audio effects
 - Interactive learning activities
 - Local storage
 - Role-based interfaces
-

2.5 Lucide Icons

The application loads Lucide icons using:

```
<script src="https://unpkg.com/lucide@latest"></script>
```

These icons are used throughout navigation, cards, forms, dashboards, buttons, and other interface elements.

2.6 Google Fonts

The project uses:

- Fredoka
- Quicksand
- Bubblegum Sans

The fonts are imported through Google Fonts and are selected specifically for a child-friendly educational visual identity.

3. Project Structure

```
Kids Learning Platform/
```

```
|— index.html
|— about.html
|— courses.html
|— teachers.html
|— events.html
|— blog.html
|— contact.html
|— login.html
|— signup.html
|— dashboard.html
|— 404.html
|
|— assets/
|   |
|   |— css/
|   |   └─ style.css
|   |
|   |— js/
|   |   └─ data.js
|   |   └─ main.js
|   |   └─ modals.js
|   |   └─ dashboard.js
|   |   └─ interactive-pad.js
|   |
|   └─ images/
|       └─ hero-banner.webp
|       └─ mascot.webp
|       └─ about-mission.webp
|       └─ about-story.webp
|       └─ about-vision.webp
|       └─ course-1.webp
|       └─ course-2.webp
|       └─ course-3.webp
|       └─ course-4.webp
|       └─ course-5.webp
|       └─ course-6.webp
|       └─ course-7.webp
|       └─ course-8.webp
|       └─ course-9.webp
|       └─ course-10.webp
|       └─ teacher-1.webp
```

```
|      |— teacher-2.webp
|      |— teacher-3.webp
|      |— teacher-4.webp
|      |— teacher-5.webp
|      |— teacher-6.webp
|      |— teacher-7.webp
|      |— teacher-8.webp
|      |— teacher-9.webp
|      |— teacher-10.webp
|      |— teacher-11.webp
|      |— teacher-12.webp
|      |— event images
|      |— gallery images
|      |— blog images
|      |— toy images
|      |— user-avatar.webp
|
|— README.md
```

3.1 File Responsibility

File	Responsibility
index.html	Main academy landing page
about.html	Academy information, story, mission and vision
courses.html	Courses and educational programs
teachers.html	Faculty and educator profiles
events.html	School events and activities
blog.html	Educational articles and updates
contact.html	Contact and academy information
login.html	Portal login

<code>signup.html</code>	Account registration
<code>dashboard.html</code>	Role-based dashboard
<code>404.html</code>	Error / fallback page
<code>style.css</code>	Global custom CSS and animations
<code>data.js</code>	Centralized application data
<code>main.js</code>	Shared UI and global application functionality
<code>modals.js</code>	Cart, search, login and content modals
<code>dashboard.js</code>	Role-based dashboard functionality
<code>interactive-pad.js</code>	Interactive educational learning pad

4. Public Website Pages

4.1 Home Page

The Home page acts as the primary entry point to the Stackly Kids platform.

Major sections include:

- Hero banner
- Academy introduction
- Course highlights
- Faculty highlights
- Events
- Testimonials
- Educational activities
- STEM learning resources
- Social media links
- Newsletter subscription
- Footer navigation

The home page also integrates the academy's visual branding and interactive elements.

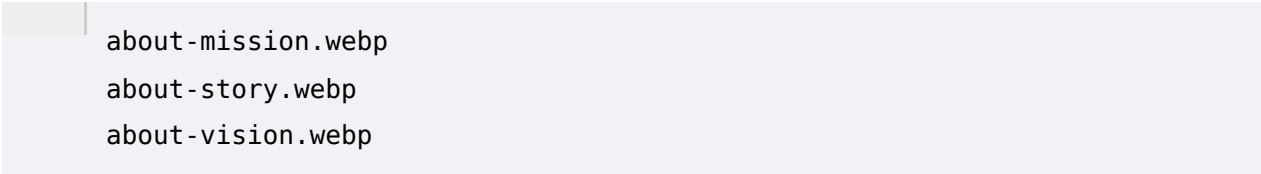
4.2 About Page

The About page presents the academy's identity and educational philosophy.

It includes:

- Academy story
- Mission
- Vision
- Educational approach
- Academy information
- Supporting imagery

The project includes dedicated image assets:



```
about-mission.webp  
about-story.webp  
about-vision.webp
```

4.3 Courses Page

The Courses page displays available educational programs.

The centralized course dataset contains **10 courses**.

Examples include:

- Math League
- Art Club
- Music Class
- Orchestra Player
- Engineering Class
- Painting Class
- Computer Programming
- Western Dance Class
- Kids Playing Club
- Literary Magazine Club

Each course contains structured information such as:

- Course ID

- Course title
 - Price
 - Original price
 - Category
 - Image
 - Rating
 - Review count
 - Description
 - Duration
 - Age group
 - Enrollment count
 - Instructor
 - Badge
-

4.4 Teachers Page

The Teachers page provides information about academy faculty.

The project contains **12 teacher profiles**.

Teacher data includes roles such as:

- Senior Educator
- Language Instructor
- Classroom Lead
- Montessori Specialist
- Creative Art Teacher
- STEM Coordinator
- Assistant Instructor
- Mathematics Educator
- Physical Education
- Rhythm Master
- Cultural Mentor

Teacher images are maintained in the assets directory:

```
teacher-1.webp  
teacher-2.webp
```

...
teacher-12.webp

4.5 Events Page

The Events page displays academy activities and community events.

The project contains **8 event records**, including:

- Teacher vs. Student Day
- Community Get Together
- Winter Feast Family Event
- Dinner Night Event
- Game Day Event
- Earth Day Party
- Birthday Party Night
- Drama Production Event

Event information is managed through the centralized data system and can be displayed using event cards and modal interfaces.

4.6 Blog Page

The Blog page is used for educational content and academy updates.

The project contains dedicated blog image assets:

blog-1.webp
blog-2.webp
blog-3.webp

Typical content areas include:

- Parent resources
 - Children's learning
 - Educational activities
 - Academy news
 - Learning tips
-

4.7 Contact Page

The Contact page provides communication information for parents and visitors.

Typical functionality includes:

- Contact information
- Location information
- Contact form
- Academy information
- Social media links
- Newsletter subscription

The project includes:

```
contact-banner.webp
```

for the contact page visual presentation.

5. Authentication Module

The project includes dedicated authentication pages:

```
login.html  
signup.html
```

5.1 Login

The login interface provides:

- Email input
- Password input
- Login submission
- Role selection / role-based access behavior
- Portal redirection

The login form uses:

```
handleLoginSubmit(event)
```

5.2 Registration

The signup page provides:

- Account creation
- User information fields
- Password fields

- Registration submission

The registration form uses:

```
handleSignupSubmit(event)
```

5.3 Client-Side Session Management

The project uses browser `localStorage` for demonstration-level session persistence.

Stored values include:

```
stackly_logged_in  
stackly_user_role  
stackly_user_email  
stackly_cart
```

The dashboard determines the active role using:

```
localStorage.getItem('stackly_user_role')
```

6. Role-Based Dashboard Architecture

The dashboard is one of the most significant functional components of the project.

The application supports three roles:

```
Student / Parent  
Teacher / Faculty  
Administrator
```

The dashboard is dynamically rendered according to the logged-in user's role.

6.1 Student / Parent Dashboard

The student role is labeled:

```
STUDENT / PARENT
```

The dashboard provides:

- Overview & Metrics
- Enrolled Classes
- Class Schedule

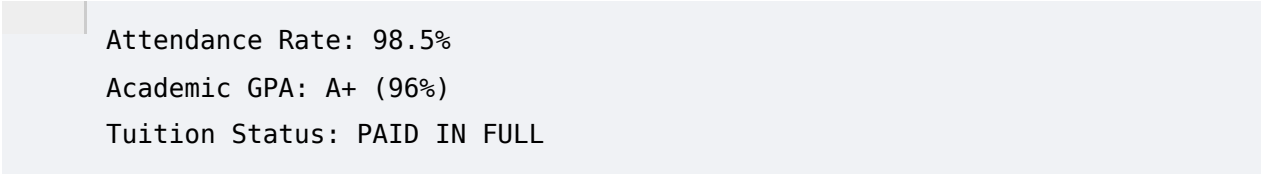
- Marks & Badges
 - STEM Toys Kit
 - Fee Receipts & Ledger
-

Student Overview

The dashboard displays educational performance metrics such as:

- Attendance rate
- Academic GPA
- Tuition status
- Subject mastery

Example metrics implemented in the project include:



Attendance Rate: 98.5%

Academic GPA: A+ (96%)

Tuition Status: PAID IN FULL

Enrolled Classes

The Student dashboard provides enrolled class information such as:

- STEM Robotics Lab
 - Montessori Math Blocks
 - Mentor
 - Classroom
 - Course progress
-

Class Schedule

The schedule view displays:

- Class time
 - Subject
 - Room
 - Current status
 - Upcoming classes
-

Marks & Badges

The platform provides a gamified achievement system.

Examples include:

- STEM Master
- Music Solo
- 100% Punctual
- Art Virtuoso

This feature encourages student engagement through visual achievement badges.

STEM Toys Kit

The Student dashboard includes access to STEM learning kit information.

The associated toy catalog is maintained through:

```
TOYS_DATA
```

Fee Receipts & Ledger

The student dashboard also provides fee-related information and payment status for demonstration purposes.

6.2 Teacher / Faculty Dashboard

The teacher role is labeled:

```
MONTESSORI EDUCATOR
```

Teacher dashboard functions include:

- Educator Overview
 - Classroom Roll Call
 - Lesson Worksheets
 - Gradebook Entry
 - Class Announcements
-

Classroom Roll Call

The project maintains an interactive attendance state:

```
window.attendanceState
```

Teachers can toggle attendance records through:

```
toggleAttendance(id)
```

The dashboard re-renders after an attendance change.

Lesson Worksheets

The teacher interface provides a dedicated area for lesson and worksheet-related activities.

Gradebook

The gradebook interface is designed for teacher-side academic record management.

Class Announcements

Teachers can access a dedicated interface for classroom announcements.

6.3 Admin Dashboard

The administrator role is labeled:

```
EXECUTIVE ACADEMY DIRECTOR
```

Admin dashboard sections include:

- Executive KPI Metrics
 - Student Admissions Roster
 - Revenue & Fee Ledger
 - Broadcast Campus Notice
 - Faculty Staff Directory
-

Executive KPI Metrics

The administrator dashboard provides high-level academy performance information.

Student Admissions

Provides a student roster and admission-related management interface.

Revenue & Fee Ledger

Provides administrative financial tracking functionality for academy fees and revenue.

Campus Announcements

Administrators can access the broadcast notice area for academy-wide communication.

Faculty Directory

Provides administrative access to faculty and staff information.

7. Dashboard JavaScript Architecture

The main dashboard functionality is implemented in:

```
assets/js/dashboard.js
```

Important functions include:

```
getDisplayName()  
switchDashTab()  
toggleMobileDashSidebar()  
toggleAttendance()  
logoutUser()  
renderRoleDashboard()
```

Role Detection

The current role is retrieved from local storage:

```
window.currentRole =  
    localStorage.getItem('stackly_user_role') || 'student';
```

This allows the same dashboard page to dynamically display different interfaces.

Dashboard Tab Switching

The function:

```
switchDashTab(tabId)
```

changes the current dashboard section and triggers a new dashboard render.

Dashboard Rendering

The main rendering function is:

```
renderRoleDashboard()
```

This function:

1. Identifies the current role.
2. Loads the relevant user data.
3. Creates role-specific sidebar navigation.
4. Validates the active tab.
5. Generates the appropriate dashboard content.
6. Updates the dashboard root element.

8. Centralized Data Architecture

The project uses:

```
assets/js/data.js
```

as a centralized data store.

This is an important architectural decision because it separates content from UI rendering.

8.1 Course Data

The project contains 10 course objects.

Each object stores:

```
id
title
price
originalPrice
startDate
category
image
rating
reviewsCount
description
duration
ageGroup
enrolledCount
instructor
badge
```

8.2 Teacher Data

Teacher information is maintained centrally and includes profile information and associated images.

8.3 Event Data

The event dataset contains 8 events with event-specific information and images.

8.4 Toy Data

The project contains 8 STEM toy image assets and a corresponding toy data structure used by the shopping cart system.

9. Shopping Cart Module

The application includes a shopping cart system for educational/STEM toys.

The functionality is implemented in:

```
assets/js/modals.js
```

9.1 Cart Storage

Cart information is persisted using:

```
localStorage.setItem(  
    'stackly_cart',  
    JSON.stringify(window.cartItems)  
);
```

9.2 Add to Cart

The function:

```
addToCart(toyId)
```

performs the following:

1. Finds the selected toy.
2. Checks whether the toy already exists in the cart.
3. Increases quantity if it exists.
4. Adds a new item if it does not exist.
5. Saves the updated cart.

6. Updates the cart badge.
 7. Displays a notification.
 8. Re-renders the cart drawer.
-

9.3 Cart Quantity Management

The function:

```
updateCartQty(toyId, delta)
```

allows users to increase or decrease item quantities.

9.4 Remove Cart Item

The function:

```
removeFromCart(toyId)
```

removes an item from the shopping cart.

10. Modal System

The modal system is implemented in:

```
assets/js/modals.js
```

Available modal functionality includes:

```
openCartDrawer()  
closeCartDrawer()  
  
openSearchModal()  
closeSearchModal()  
  
openLoginModal()  
closeLoginModal()  
  
quickLogin()  
  
openCourseModal()  
  
openEventModal()
```

These functions provide reusable overlays for:

- Shopping cart
 - Search
 - Login
 - Course details
 - Event details
-

11. Search Functionality

The project includes a search interface managed by:

```
openSearchModal()  
closeSearchModal()  
performSearch()
```

The search system provides a centralized interface for navigating or finding content within the platform.

12. Interactive Learning Pad

One of the unique features of the project is the Interactive Learning Pad.

The implementation is located in:

```
assets/js/interactive-pad.js
```

The learning pad provides multiple educational modes.

12.1 Drawing Mode

The drawing interface provides children with an interactive canvas-like learning environment.

It is designed to encourage:

- Creativity
 - Hand-eye coordination
 - Drawing practice
 - Interactive learning
-

12.2 Soundboard

The sound mode provides interactive sound effects.

The platform uses the browser's Web Audio API for generated sound effects.

12.3 ABC Learning

The platform includes an ABC learning activity for early education.

The primary function is:

```
handleAbcClick()
```

12.4 Number Learning

The learning pad also supports basic number learning activities.

12.5 Balloon Pop Mini Game

The platform includes a simple interactive balloon-pop game designed as an educational entertainment feature.

13. Sound Effects System

The global sound system is implemented in:

```
assets/js/main.js
```

The project uses the Web Audio API.

Supported sound effects include:

```
playPop()  
playChime()  
playSuccess()  
playWhistle()  
playDrum()
```

Sound Toggle

Users can enable or disable sound effects through:

```
toggleSoundFX()
```

The current state is controlled by:

```
window.soundEnabled
```

14. Toast Notification System

The project implements a global toast notification system through:

```
window.showToast()
```

It is used to provide immediate feedback for actions such as:

- Adding an item to cart
- Removing an item
- Login actions
- Logout
- Newsletter subscription
- Other interactive operations

This improves the user experience without requiring full-page reloads.

15. Reusable UI Components

The project uses reusable JavaScript-generated components.

Examples include:

- Stackly logo
- Social media icon groups
- Toast notifications
- Cart components
- Dashboard components
- Modal interfaces

The logo is dynamically generated through:

```
renderStacklyLogo()
```

This reduces duplication across multiple pages.

16. Social Media Component

A reusable social icon component is implemented as:

```
renderSocialIconsGroup(size)
```

The component supports:

- Facebook
- X / Twitter
- Instagram
- LinkedIn

SVG icons are embedded directly into the generated markup.

17. UI / UX Design

17.1 Visual Identity

The application uses a playful educational design appropriate for children and parents.

The primary visual characteristics include:

- Rounded cards
 - Bright accent colors
 - Friendly typography
 - Illustrated imagery
 - Soft backgrounds
 - Large interactive controls
 - Playful animations
-

17.2 Typography

The application uses:

Fredoka

Used primarily for:

- Headings
- Dashboard titles
- Child-oriented content

Quicksand

Used primarily for:

- Body text
- Descriptions
- Forms
- Navigation

Bubblegum Sans

Used for playful branding elements.

18. Responsive Design

The platform is designed to support multiple screen sizes.

Desktop

- Full navigation
- Multi-column layouts
- Expanded dashboard sidebar
- Large course and teacher cards

Tablet

- Flexible grid
- Reduced spacing
- Adaptive navigation
- Responsive dashboard layout

Mobile

- Collapsible navigation
 - Single-column cards
 - Mobile dashboard sidebar
 - Touch-friendly controls
 - Responsive forms
-

19. CSS Architecture

The custom stylesheet is:

```
assets/css/style.css
```

The stylesheet contains:

- Font definitions
- Global body styling
- Scrollbar customization
- Background patterns
- Animations

- Component-specific styles
 - Responsive behavior
-

Custom Animations

The CSS includes animations such as:

```
float
float-reverse
wiggle
```

These animations are used to create a playful and dynamic visual experience.

20. Image and Asset Strategy

The project uses WebP image assets throughout the platform.

Major asset categories include:

```
Course Images
Teacher Images
Event Images
Gallery Images
Blog Images
Toy Images
About Images
Hero Images
Testimonials
Social/Instagram Images
```

Using WebP helps reduce image file size while maintaining visual quality.

21. Authentication and Security

The current project is a frontend prototype.

Authentication state is maintained through browser local storage.

Important storage keys include:

```
stackly_logged_in
stackly_user_role
stackly_user_email
```

Current Limitations

- No backend authentication server.
- No database.
- No encrypted password storage.
- No server-side session validation.
- Local storage can be modified by the client.
- Role information is client-controlled.

Therefore, the current authentication implementation should **not be considered production-grade authentication**.

22. Data Storage

The project uses two primary approaches.

Static JavaScript Data

Educational content is maintained in:

```
assets/js/data.js
```

Browser Local Storage

Used for:

- Authentication state
- User role
- User email
- Shopping cart
- Session-related information

This architecture is suitable for a frontend demonstration or prototype.

23. Error Handling

The project contains:

```
404.html
```

for invalid or unavailable pages.

The application also uses defensive JavaScript patterns such as checking whether DOM elements exist before manipulating them.

The audio functionality includes `try/catch` blocks to prevent audio-related browser errors from breaking the application.

24. Testing Summary

24.1 Functional Testing

The following areas should be tested:

- Home navigation
- About page
- Course browsing
- Teacher browsing
- Event browsing
- Blog navigation
- Contact page
- Login
- Registration
- Dashboard role switching
- Student dashboard
- Teacher dashboard
- Admin dashboard
- Shopping cart
- Search
- Course modal
- Event modal
- Toast notifications
- Interactive learning pad
- Sound controls

24.2 Role-Based Testing

Student

Verify:

- Student login
- Student overview
- Classes

- Schedule
- Marks
- STEM toys
- Fees
- Logout

Teacher

Verify:

- Teacher login
- Educator overview
- Attendance
- Lessons
- Gradebook
- Announcements
- Logout

Admin

Verify:

- Admin login
- KPI dashboard
- Admissions
- Finance
- Announcements
- Faculty
- Logout

25. Browser Compatibility

The application is designed for modern browsers.

Browser	Expected Support
Google Chrome	Yes
Microsoft Edge	Yes

Mozilla Firefox	Yes
Safari	Yes
Mobile Chrome	Yes
Mobile Safari	Yes

Because the project uses modern JavaScript, Tailwind CDN, Web Audio API, localStorage, and modern CSS features, testing should focus on current browser versions.

26. Performance Considerations

The project uses several positive performance practices:

- WebP image assets
- Centralized data
- Reusable JavaScript components
- CDN-based Tailwind CSS
- SVG icons
- Client-side rendering
- Local browser storage

Recommended Production Optimizations

- Compress all images further where appropriate.
 - Use a production Tailwind build instead of the CDN.
 - Minify CSS and JavaScript.
 - Implement lazy loading for below-the-fold images.
 - Use a CDN for static assets.
 - Remove unused assets.
 - Implement caching headers.
 - Reduce unnecessary DOM re-rendering.
-

27. Deployment Process

The project includes Git repository metadata, indicating that Git is part of the development workflow.

Typical deployment commands are:

```
git add .  
  
git commit -m "Project Update"  
  
git push origin main
```

For local development, the project can be served through a local HTTP server:

```
python -m http.server 5500
```

Then open:

```
http://127.0.0.1:5500
```

28. Production Architecture Recommendation

The current application is primarily a frontend prototype. For production deployment, the architecture can be extended to:

```
Frontend  
  |  
  ├── HTML  
  ├── CSS  
  ├── JavaScript  
  |  
  ▼  
REST API  
  |  
  ├── Authentication  
  ├── Student Management  
  ├── Teacher Management  
  ├── Course Management  
  ├── Event Management  
  ├── Attendance  
  ├── Grades  
  ├── Fees  
  └── Orders
```

29. Recommended Production Features

For a complete production version, the following features can be added:

- Secure backend authentication
- Parent/student account management
- Teacher account management
- Admin role authorization
- Database-backed courses
- Online class management
- Attendance database
- Grade management
- Fee payment integration
- Online learning content
- Real-time notifications
- Email notifications
- Parent-teacher messaging
- Order processing for STEM toys
- Payment gateway
- Cloud image storage
- Analytics

30. Security Recommendations

For production implementation:

- Use HTTPS.
- Hash passwords using a secure password-hashing algorithm.
- Never store passwords in localStorage.
- Implement server-side authentication.
- Use secure HTTP-only cookies for sessions.
- Validate all input on the server.
- Implement role-based authorization on the backend.

- Protect dashboard API endpoints.
 - Sanitize user-generated content.
 - Implement CSRF protection where applicable.
 - Apply rate limiting to authentication endpoints.
 - Maintain audit logs for administrative operations.
-

31. Future Enhancements

The platform can be expanded with:

Learning Management System

- Online lessons
- Video classes
- Assignments
- Quizzes
- Progress tracking

Parent Portal

- Child progress
- Attendance
- Fee payments
- Teacher communication
- Notifications

Teacher Portal

- Lesson planning
- Attendance
- Grade management
- Student reports

Admin Portal

- Admissions
- Faculty management
- Financial reports
- Course management
- Analytics

Interactive Learning

- More educational games
 - Drawing activities
 - Alphabet exercises
 - Mathematics games
 - Interactive stories
 - Audio-based learning
-

32. Project Strengths

The project has several notable technical strengths:

1. Clear multi-page structure

Each major public feature has its own HTML page.

2. Centralized data management

Course, teacher, event, and other data are separated from presentation logic.

3. Role-based dashboard

A single dashboard architecture supports Student, Teacher, and Admin roles.

4. Reusable components

Shared elements such as logos, social icons, modals, and notifications reduce duplication.

5. Interactive learning features

The project goes beyond a standard educational website by providing drawing, sound, ABC, number, and mini-game functionality.

6. Client-side persistence

localStorage provides persistent prototype state for login information and shopping cart data.

7. Responsive design

Tailwind CSS and custom CSS provide responsive behavior across device sizes.

8. Child-focused UX

The use of playful fonts, colors, animations, illustrations, and interactive sounds creates an age-appropriate experience.

33. Technical Limitations

The following limitations were identified from the project implementation:

- The application does not currently have a backend server.
 - Data is primarily static JavaScript data.
 - Authentication is client-side.
 - Dashboard information is demonstration data.
 - localStorage is used for session and cart state.
 - Social media links currently route to the 404 page rather than actual social profiles.
 - Newsletter submission is demonstrated through a toast notification rather than a backend subscription service.
 - Payment processing is not implemented.
 - Course enrollment is not connected to a real LMS.
 - Attendance and gradebook information are not persisted to a server database.
-

34. Conclusion

Stackly Kids Learning Platform is a comprehensive frontend educational website that combines a public preschool/academy website with an interactive role-based learning portal.

The project provides a strong foundation for a future Learning Management System by combining:

- Educational content
- Course management
- Faculty information
- School events
- Blog content
- Student dashboard
- Teacher dashboard
- Admin dashboard
- Interactive learning tools
- STEM toy shopping functionality
- Responsive UI
- Reusable JavaScript components

The current implementation is well suited as a frontend prototype, demonstration project, or academic project. With the addition of a secure backend, database, real

authentication, payment processing, and persistent academic records, it can be developed into a complete production-ready education management platform.

35. Developer Information

Project Name: Stackly Kids Learning Platform

Application Name: Stackly Salem Preschool & Academy

Developer: Dhamodharan P

Role: Frontend Developer

Architecture: Static Multi-Page Web Application with Client-Side Role-Based Dashboard

Frontend: HTML5, CSS3, JavaScript ES6

CSS Framework: Tailwind CSS

Icons: Lucide Icons

Fonts: Fredoka, Quicksand, Bubblegum Sans

Storage: Browser localStorage for prototype state

Image Format: WebP

Deployment: Static Web Hosting / GitHub Pages Compatible

Version: 1.0

Document Version: 1.0